

WEEK – 2

DAA lab assignment

Q-1 take random input of thousand numbers and perform comparison of linear and binary search?

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#define N 1000

int cmp(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}

int linear(int a[], int n, int x) {
    for (int i = 0; i < n; i++)
        if (a[i] == x) return i;
    return -1;
}

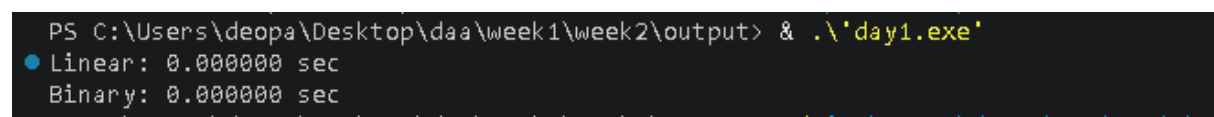
int binary(int a[], int n, int x) {
    int l = 0, h = n - 1;
    while (l <= h) {
        int m = (l + h) / 2;
        if (a[m] == x) return m;
        if (a[m] < x) l = m + 1;
        else h = m - 1;
    }
    return -1;
}
```

```

}

int main() {
    int a[N], x;
    clock_t s;
    srand(time(0));
    for (int i = 0; i < N; i++) a[i] = rand() % 10000;
    qsort(a, N, sizeof(int), cmp);
    x = a[rand() % N];
    s = clock();
    linear(a, N, x);
    printf("Linear: %.6f sec\n",
        (double)(clock() - s) / CLOCKS_PER_SEC);
    s = clock();
    binary(a, N, x);
    printf("Binary: %.6f sec\n",
        (double)(clock() - s) / CLOCKS_PER_SEC);
    return 0;
}

```



```

PS C:\Users\deopa\Desktop\daa\week1\week2\output> & .\'day1.exe'
Linear: 0.000000 sec
Binary: 0.000000 sec
PS C:\Users\deopa\Desktop\daa\week1\week2\output>

```

because both operations are extremely fast for only 1000 elements. For a **proper performance comparison**, repeat each search many times (e.g. 100,000 times).

```

int cmp(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}

```

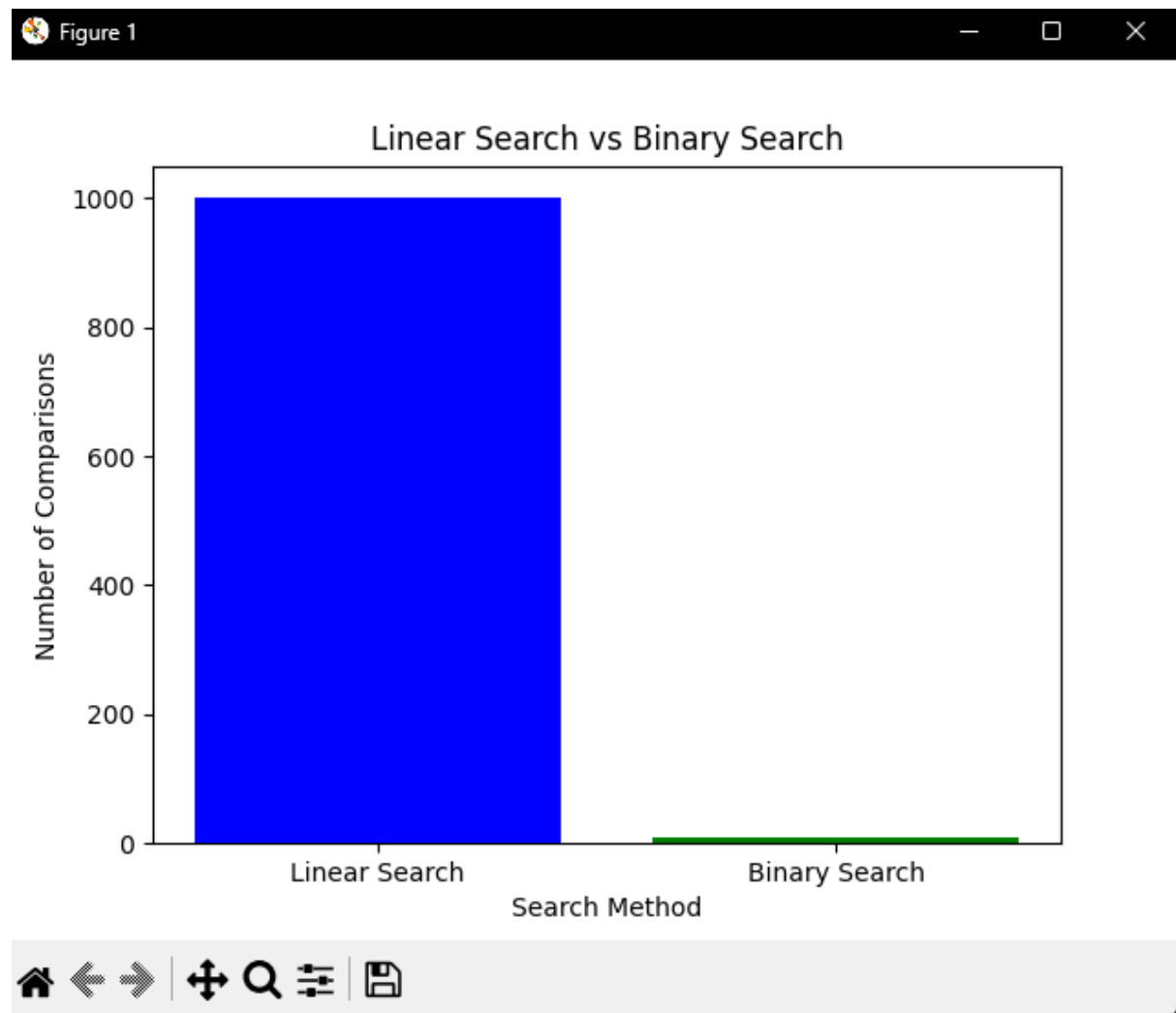
This function is used by qsort() to sort the array in **ascending order**.

- void * can point to any data type.
- (int *)a converts it to an integer pointer.

- `*(int *)a` gets the actual integer value.

Binary Search is faster than Linear Search because Binary Search divides the sorted array into halves at every step, while Linear Search checks each element one by one. Therefore, Binary Search has a time complexity of $O(\log n)$ compared to $O(n)$ for Linear Search.

For 1000 elements, binary search generally requires much fewer comparisons and is therefore more efficient.



QUES-2-Write a C program to generate 1000 random numbers and compare the execution time of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. Display the time taken by each sorting algorithm and plot a comparison graph using Python.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#define N 1000

void bubble(int a[]) {
    for(int i=0;i<N-1;i++)
        for(int j=0;j<N-i-1;j++)
            if(a[j]>a[j+1]) {
                int t=a[j]; a[j]=a[j+1]; a[j+1]=t;
            }
}

void selection(int a[]) {
    for(int i=0;i<N-1;i++) {
        int m=i;
        for(int j=i+1;j<N;j++)
            if(a[j]<a[m]) m=j;
        int t=a[i]; a[i]=a[m]; a[m]=t;
    }
}

void insertion(int a[]) {
    for(int i=1;i<N;i++) {
        int x=a[i],j=i-1;
        while(j>=0 && a[j]>x)
            a[j+1]=a[j],j--;
        a[j+1]=x;
    }
}
```

```
}
```

```
void merge(int a[],int l,int r) {  
    if(l>=r) return;  
    int m=(l+r)/2,t[N],i=l,j=m+1,k=l;  
    merge(a,l,m); merge(a,m+1,r);  
    while(i<=m && j<=r) t[k++]=a[i]<a[j]?a[i++]:a[j++];  
    while(i<=m)t[k++]=a[i++];  
    while(j<=r)t[k++]=a[j++];  
    for(i=l;i<=r;i++)a[i]=t[i];  
}
```

```
void quick(int a[],int l,int r) {  
    if(l>=r)return;  
    int i=l,j=r,p=a[(l+r)/2],t;  
    while(i<=j) {  
        while(a[i]<p)i++;  
        while(a[j]>p)j--;  
        if(i<=j)t=a[i],a[i++]=a[j],a[j--]=t;  
    }  
    quick(a,l,j); quick(a,i,r);  
}
```

```
int main() {  
    int a[N],b[N];  
    clock_t s;  
    srand(time(0));  
    for(int i=0;i<N;i++) a[i]=rand()%10000;  
    for(int x=0;x<5;x++) {  
        for(int i=0;i<N;i++) b[i]=a[i];
```

```

s=clock();

if(x==0) bubble(b);

if(x==1) selection(b);

if(x==2) insertion(b);

if(x==3) merge(b,0,N-1);

if(x==4) quick(b,0,N-1);

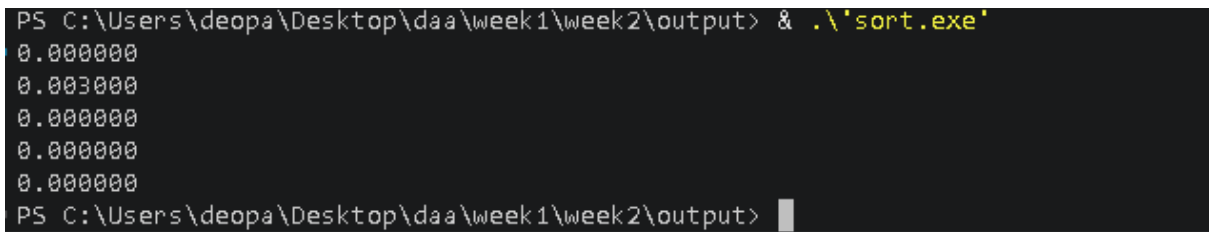
printf("%.6f\n",(double)(clock()-s)/CLOCKS_PER_SEC);

}

return 0;

}

```



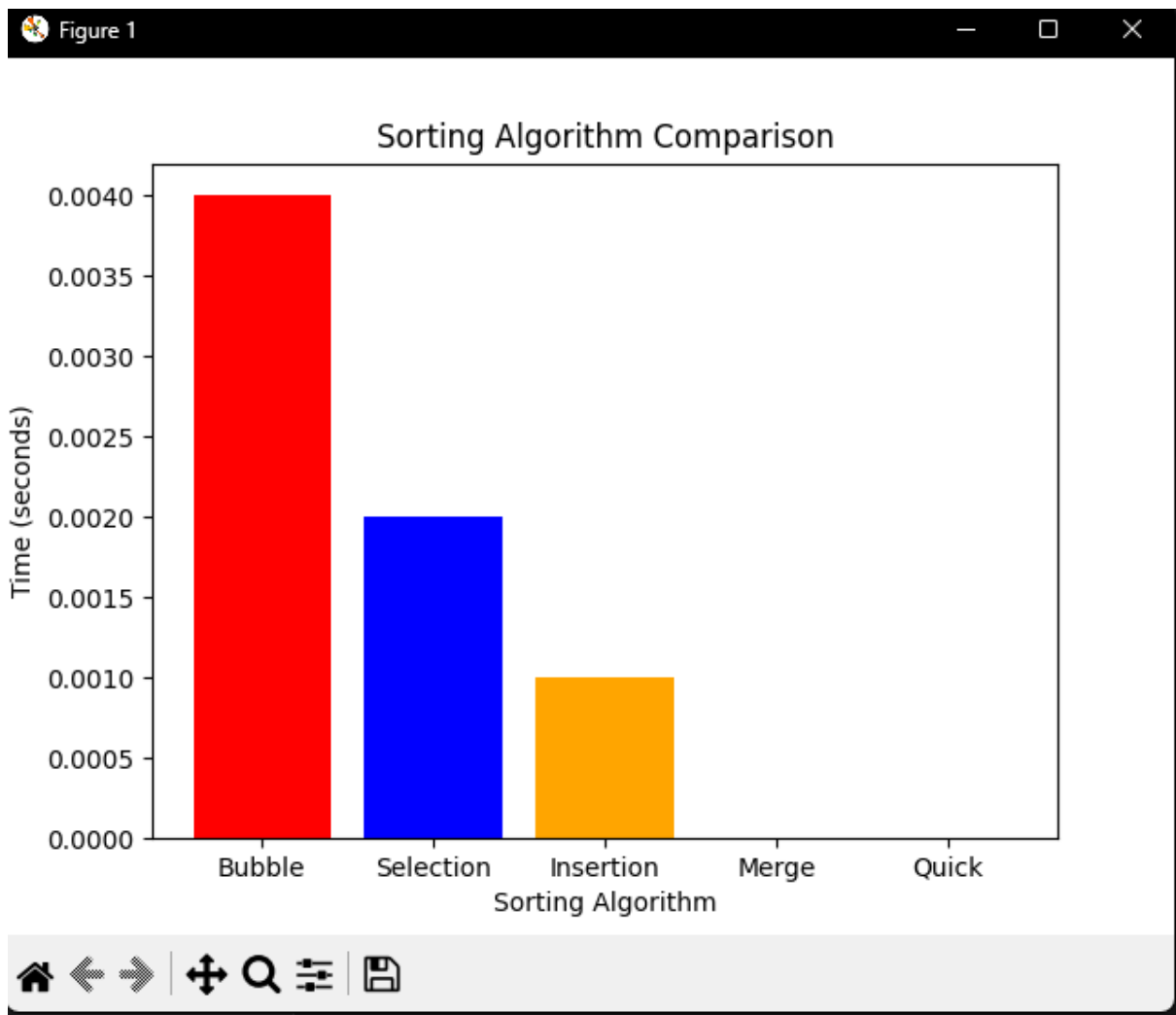
```

PS C:\Users\deopa\Desktop\daa\week1\week2\output> & .\'sort.exe\'
0.000000
0.003000
0.000000
0.000000
0.000000
0.000000
PS C:\Users\deopa\Desktop\daa\week1\week2\output>

```

Conclusion

The experiment shows that **Merge Sort and Quick Sort are generally faster** than Bubble Sort, Selection Sort, and Insertion Sort for a large number of random elements. Bubble Sort, Selection Sort, and Insertion Sort have **$O(n^2)$** time complexity, while Merge Sort has **$O(n \log n)$** and Quick Sort has an average complexity of **$O(n \log n)$** . Therefore, **Merge Sort and Quick Sort are more efficient for large datasets.**



Time Complexity Comparison Table

Sorting Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$